

Course 2 Notes: Advanced Learning Algorithms

Andrew Ng — Jesse Hernandez

DeepLearning.AI/Stanford Online — 2025

My notes for the second course in a three part machine learning specialization course on coursera consisting of four learning modules.

- Course objectives

- Introduction to Neural Networks
- Neural Network Training
- Practical advice on building ML systems
- Decision Trees

1. Week 1: Neural Networks

1.1 Overview

- Key objectives

- Introduction to Neural Networks
- Neural Network Inference (Predictions)

- Background context

- Building Neural network from scratch and with Tensorflow for inference and training

1.1.1 Neurons as inspiration

Note

- The origins of neural networks were to build algorithms that mimicked the brain
- Re-popularized in 2005 in the category of Deep Learning
- Initially successful in speech recognition then image recognition followed by Natural Language Processing (NLP) and now used in many more applications
- Biological neurons function by taking many electrical impulses as inputs and outputs the response to send to the next neuron until it reaches the brain
- Neural Networks saw huge performance improvements particularly with Big Data, e.g., performance scales with amount of data,

1.2 Introduction to Neural Networks

Definition

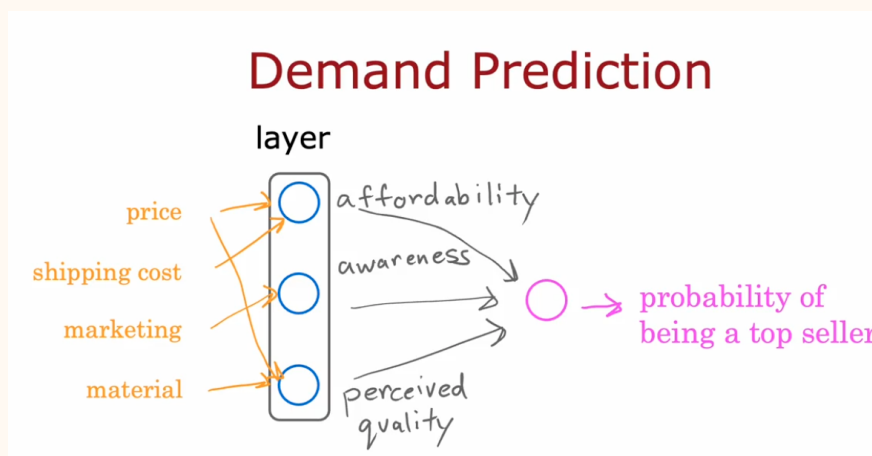
A **Neural Network** is a type of machine learning inspired by how the brain works using layered structure of interconnected nodes or 'neurons' to process data, recognize patterns and make predictions.

Example

Demand Prediction example using neural network.

- We are trying to predict the probability of being a top seller for clothing product
- The inputs or features to the model are price, shipping cost, marketing, and material (input layer)
- The criteria (**activations**) are affordability, awareness, and perceived quality (forms the nodes of our neural layer or **hidden layer** that outputs **activation values**)
- Relevant features map out to different nodes to access its criteria ranking, in practice the network will find the best features to use (analogous to automated feature engineering)
- All outputs from the layer is fed to another node (output layer) to process the probability of being a top seller
- the independent nodes each run independent logistic regression (input model) algorithms to output probabilities
- Simple to vectorize inputs and outputs of to run through layers and come to a prediction, here $\vec{x}$, $\vec{a}$ are the inputs and activation values respectively,

$$\vec{x} \rightarrow (\text{hidden layer}) \rightarrow \vec{a} \rightarrow (\text{output layer}) \rightarrow a$$



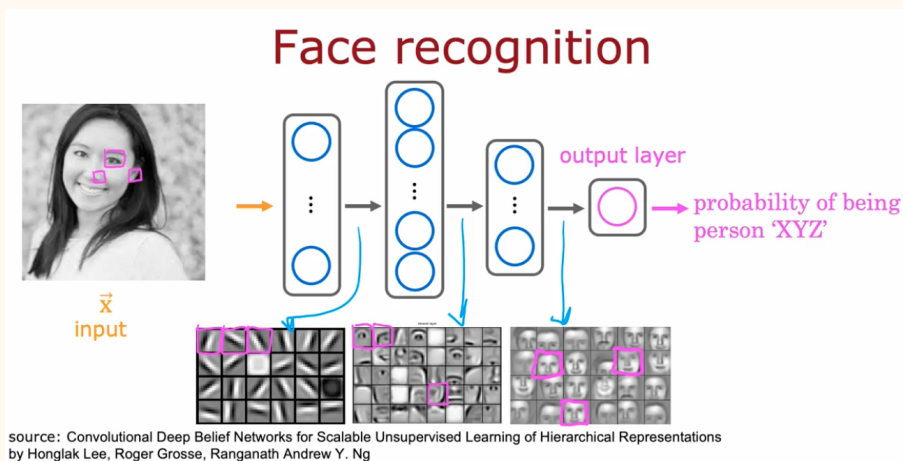
Note

- Depending on the **neural network architecture** there can be many hidden layers with different number of nodes in each (**multilayer perceptron**)
- choosing how to construct the neural network depending on what problem you have can affect performance and accuracy and will be discussed in week 3

1.2.1 Recognizing Images**Example**

Face recognition problem is to identify the person in the image using the pixel data of the image

- The image data is read as a $m \times n$ matrix of pixel intensities
- The input features are then these values rolled out into a vector of $m \times n$ length, e.g., a 1000x1000 pixel image will have 1 million pixel intensity features as input to the network
- Shown below is how many hidden layers may naturally break the task into pieces at each layer with different image scopes and aggregate the information to get a probability for being person XYZ
 1. Layer 1 may look for edges or shapes (fine scope)
 2. Layer two may take layer 1 input to find facial structures, e.g., eyes, nose, etc (medium scope)
 3. Layer 3 may aggregate the facial features to form full facial images (large scope)

**Note**

The neural network can be tasked to find cars and the layers will automatically learn to search for car features.

1.3 Neural Networks in action

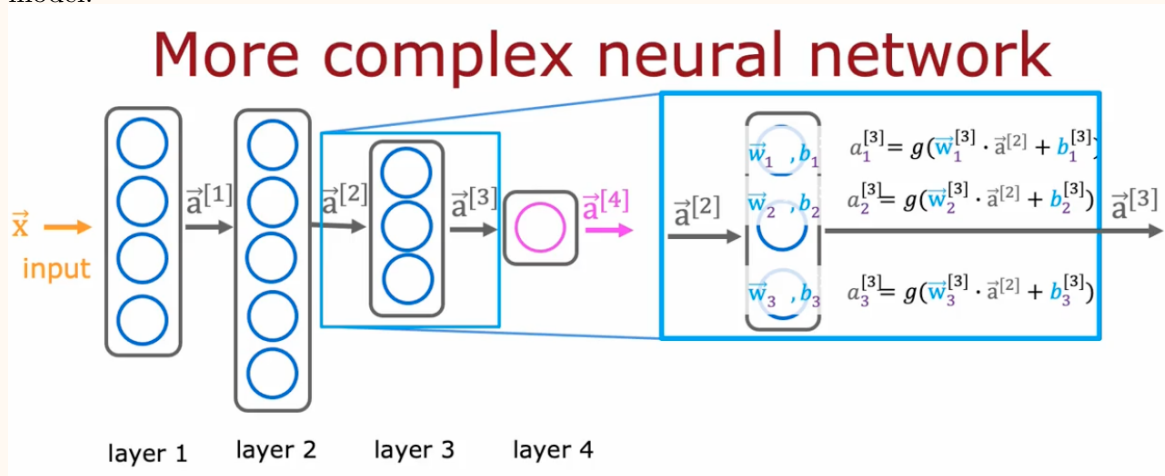
Definition

The **neural network layer** is composed of many nodes ($n_1 \dots n_i$) taking in all the features $\vec{x}$ with parameters optimized for each node ($\vec{w}_i, b_i$) from the set of features and outputs the activation value for each node a_i , which forms the activation vector (hidden layer output) $\vec{a}$.

- Each node is running a logistic regression model such that $a_i = g(z)$ where the definitions for the regression model z (linear or polynomial) is transformed using a sigmoid ($g(z)$) function discussed in course 1.
- For multi-layer neural networks we will use the superscript notation $\vec{a}^{[j]}$ to denote the j^{th} layer
- The activation values of each layer $\vec{a}^{[j]}$ then become the input values for the subsequent layer

Example

An example of a more complex neutral network may look like this (below), where the calculation for the third hidden layer using a logisitic regression tranformantion of a linear regression model:



- The third layer calculations are using the second layer outputs: $a_i^{[3]} = g(\vec{w}_i^{[3]} \cdot \vec{a}^{[2]} + b_i^{[3]})$

This can be generalized for logistic regression with a linear regression model as:

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}), \quad (1)$$

where l is the layer number and j is the node number. Here $g(z)$ is the **activation function** and can take other forms but is a sigmoid function here.

1.3.1 Inference: making predictions

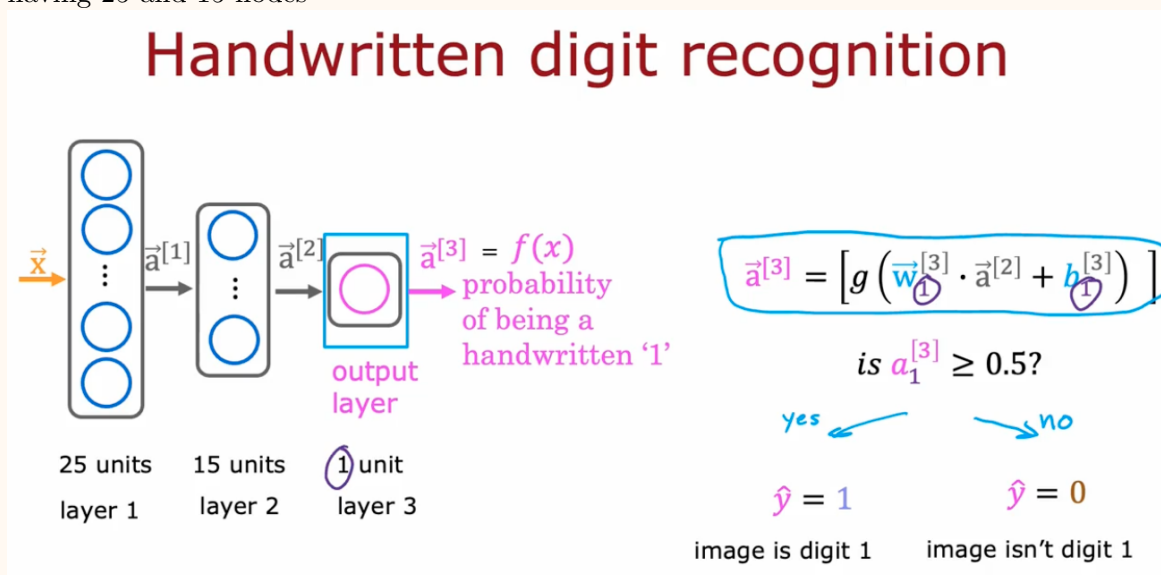
Definition

Forward propagation is a machine learning algorithm used with neural networks to make predictions by propagating the results forward through the hidden layers to the output value (see example below)

- Forward propagation can be used to predict values using an existing neural network with tuned parameters, in the next lesson we will learn to train our own neural network

Example

Handwritten digit recognition example is shown below where we want to find the probability of the image being a one and take an 8 x 8 pixel image as input with 2 hidden layers having 25 and 15 nodes



- The image above shows only the computation for the final output layer, more generally each activation vector is solved:

$$\vec{a}^{[l]} = \begin{bmatrix} g(\vec{w}_1^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}) \\ \vdots \\ g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}) \end{bmatrix} \quad (2)$$

1.4 Implementations

Course Labs

- Neurons and Layers
 - Explore inner workings of neural network structure (building on the course 1 regression) and implementation in Tensorflow and Keras.

- Simple Neural network (Tensorflow)
 - Build a small neural network using Tensorflow and explore functionality.
- Simple Neural network (Numpy)
 - Build a small neural network using Numpy, the same as the previous lab and explore from scratch implementation and comparison to production library.

Programming Assignment

- Implement neural network to recognize hand-written digits zero and one (binary)

2. Week 2: Neural Network Training

- Key objectives

- Neural Network Training
- Tensorflow efficient implementation
- Activation functions (ReLU, Softmax)
- Multiclass and multilabel classification
- Advanced Optimization (Adam)
- Introduction to Convolution Neural Network

2.1 Tensorflow implementation

Definition

Back propogation is used to compute derivatives for gradient descent

Note

The tensorflow implementation workflow to build and train a neural network is broken down in a few simple steps:

1. Using `tensorflow.keras` import functions, e.g., `Sequential`, `Dense`, `BinaryCrossEntropy`
2. Build the model using `Sequential` and `Dense` to build a sequential layer of nodes with specified activation model for nodes
3. Specify the loss function using the `compile(loss=MeanSquaredErrors())`
4. Fit the model to find the optimal parameters from the training data using `fit(X,y,epochs=100)`

```
# Import libraries
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.losses import BinaryCrossEntropy

# Make the neural network model with activation model
model = Sequential( [
    Dense(units=25, activation='sigmoid ')
    Dense(units=15, activation='sigmoid ')
    Dense(units=1, activation='sigmoid ')
] )

# Compile for a specific loss function
model.compile(loss=BinaryCrossEntropy())

# Fit the model to optimize parameters based on training data
model.fit(X,y,epochs=100)
```

2.2 Activation Functions

Definition

The **activation function** defined earlier tells us what model is used in the nodes for our neural network.

Linear activation function also considered "no activation function" takes the form:

$$g(z) = z = \vec{w} \cdot \vec{x} + b$$

Sigmoid activation function is what we already learned in the previous course and defined as

$$g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + \exp[-(\vec{w} \cdot \vec{x} + b)]} \quad (3)$$

for a linear regression.

The **ReLU (Rectified Linear Unit)** activation function is commonly used as a powerful alternative and is defined as

$$g(z) = \max(0, z) = \begin{cases} 0, & z < 0 \\ \vec{w}_j^{[i]} \cdot \vec{x} + b_{[i],j}^{[i]}, & z > 0 \end{cases} \quad (4)$$

2.2.1 Common activation functions

Note

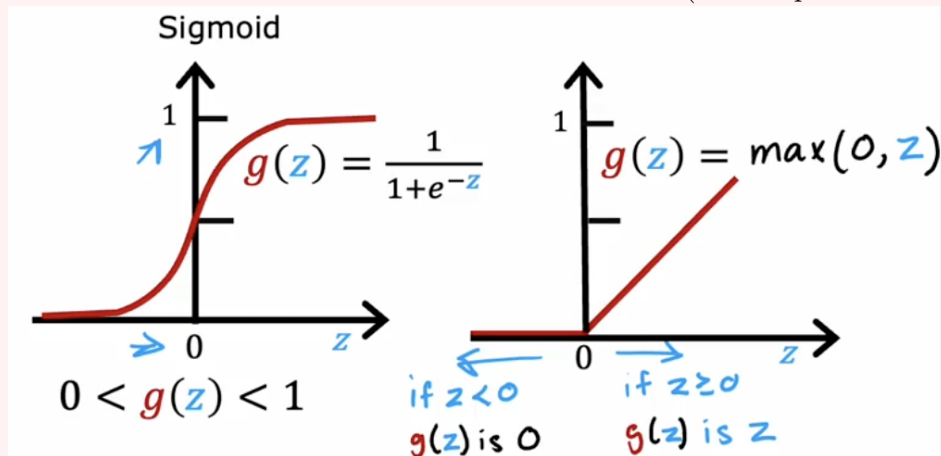
How do we choose the appropriate activation function for the problem to be solved? This is usually a natural decision based on the type of output that is required from the model or the labels from the training data y .

The choice for the output layer can be broken up in the following logic:

- Classification problems that require a binary decision would work best with the logistic activation
- For continuous numerical output, that take negative and positive numbers e.g., stock price daily gain, the linear regression activation would be the most natural choice.
- For continuous numerical output that take only positive values, the ReLU activation function would be the most natural.

How do we choose the activation function $g(z)$ for the hidden layers? The ReLU is by far the most common activation function for the hidden layers, superseded by the sigmoid activation. The benefits of the ReLU over the sigmoid function are:

- The ReLU is functionally simpler and faster to compute, therefore more efficient in the hidden layers where there are many nodes
- When doing gradient descent, the algorithm tends to be much slower where the analytic function is flat (the derivative is very small in these regions), therefore given the function form ReLU is also faster for this reason (look at picture below)

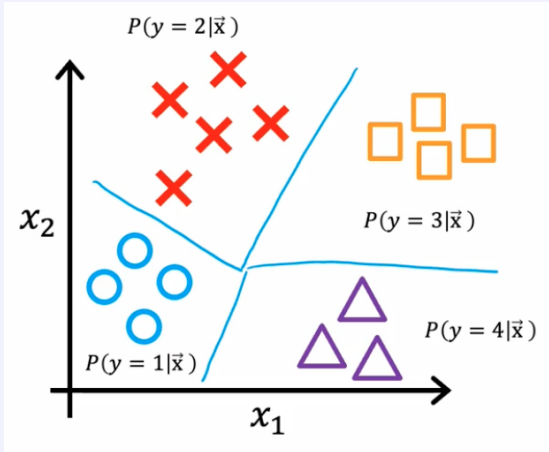


2.3 Multiclass Classification

Definition

Multiclass Classification problem is one which has more than two output labels y_i and has many different categories (exmaple image below)

- For the handwriting example we had 2 outputs for 0 or 1
- Multiclass example would be the zipcode where $y = 0 \dots 9$



2.3.1 Softmax Regression Algorithm

Definition

Softmax is a logistic regression generalization for the multiclass classification. The model used is generalized for multiple outputs (y):

$$z_i = \vec{w}_i \cdot \vec{x} + b_i \quad i = 0, \dots, N_y, \quad (5)$$

where N_y are the number of output labels. The output of the softmax regression then is generalized to:

$$a_i = \frac{e^{z_i}}{\sum_{j=1}^{N_y} e^{z_j}} = P(y = i | \vec{x}) \quad (6)$$

where the probability of the set of outputs is 1

$$\sum_{j=1}^{N_y} P(y = j | \vec{x}) = 1. \quad (7)$$

The **Cost function** is then generalized in terms of the generalized loss function called the **Crossentropy loss** (similar to the logistic loss function):

$$\text{loss}(a_1, \dots, a_N, y) = \begin{cases} -\log a_1 & \text{if } y = y_1 \\ \vdots & \\ -\log a_N & \text{if } y = y_N \end{cases} \quad (8)$$

and the cost function is the average of Eq. (8) formalism.

Note in Eq. (8) above, only the line that corresponds to the target contributes to the loss, other lines are zero. To write the cost equation we need an 'indicator function' that will be 1 when the index matches the target and zero otherwise.

$$\mathbf{1}\{y == n\} = \begin{cases} 1, & \text{if } y == n. \\ 0, & \text{otherwise.} \end{cases}$$

Now the cost is:

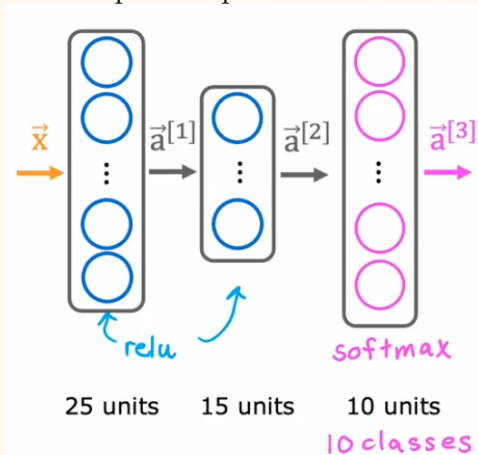
$$J(\mathbf{w}, b) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{j=1}^N \mathbf{1}\{y^{(i)} == j\} \log \frac{e^{z_j^{(i)}}}{\sum_{k=1}^N e^{z_k^{(i)}}} \right] \quad (4)$$

Where m is the number of examples, N is the number of outputs. This is the average of all the losses.

2.3.2 Softmax with Neural Networks

Example

An example of the structure of the neural network with a softmax output layer for multiple class output is represented below:



The implementation in tensorflow the neural network is the same as before with the softmax activation layer `Dense(units=N, activation='softmax')` and the loss function Eq. (8) is called `SparseCategoricalCrossentropy` and imported from `tensorflow.keras.losses`.

Note

Due to numerical rounding errors the method of implementation in tensorflow is much more accurate for softmax activation when activation function Eq. (8) is directly computed within the loss Eq. (8) and implemented like this:

```
Old:
# Make the neural network model with activation model
model = Sequential( [
Dense(units=25, activation='sigmoid')
Dense(units=15, activation='sigmoid')
Dense(units=10, activation='softmax')
] )

model.compile(loss=SparseCategoricalCrossEntropy())
...

Accurate:
# Make the neural network model with activation model
model = Sequential( [
Dense(units=25, activation='sigmoid')
Dense(units=15, activation='sigmoid')
Dense(units=10, activation='linear')
] )

# loss function
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))

# fit model
model.fit(X,y,epohks=100)

# extra steps due to linear output z instead of probability a
logit = model(X) # this is z from output
# predict
f_x = tf.nn.softmax(logit) # this is a now
```

The above extra lines of code to get the probabilities is due to changing the output layer to be 'linear' and thus outputs z_i instead of probabilities a_i that we saw with logistic regression using sigmoid activation.

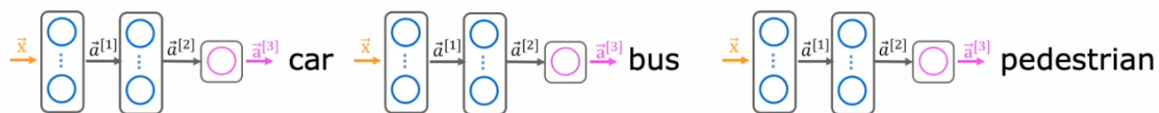
This implementation would also be slightly more accurate for logistic regression with `BinaryCrossentropy(from_logits=True)` loss as well, but not as important.

2.4 Multi-label Classification

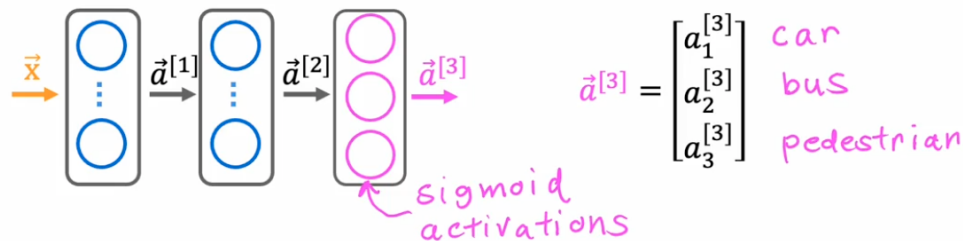
Definition

A **multi-label classification** problem is one where the output layer gives multiple labels for one output, e.g., looking for whether one image has a car, bus and pedestrian in it. and should not be mistaken for the the multiclass classification.

Multi-label Classification



Alternatively, train one neural network with three outputs



2.5 Advanced Optimization

Definition

The **Adam (Adaptive Moment estimation) algorithm** automatically readjusts the learning rate of gradient descent.

– This algorithm has a different learning rate for every parameter in the gradient descent algorithm and adapts for convergence.

- if the steps tend to move in the same direction (converging slowly in one direction) the learning rate is increases for faster convergence
- If the steps tend to oscillate the learning rate is decreases to optimize convergence

– The implementation of the Adam algorithm in tensorflow is quite simple by importing it from `tf.keras.optimizers` and calling it in the `compile()` function when training

```
model.compile(loss=[loss function],
              optimizer=Adam(learning_rate=1e-3)) #initial learning rate
```

2.6 Additional Layer Types

What we have been using is the Dense layer each neuron output is a function of **all** the activation outputs of the previous layer.

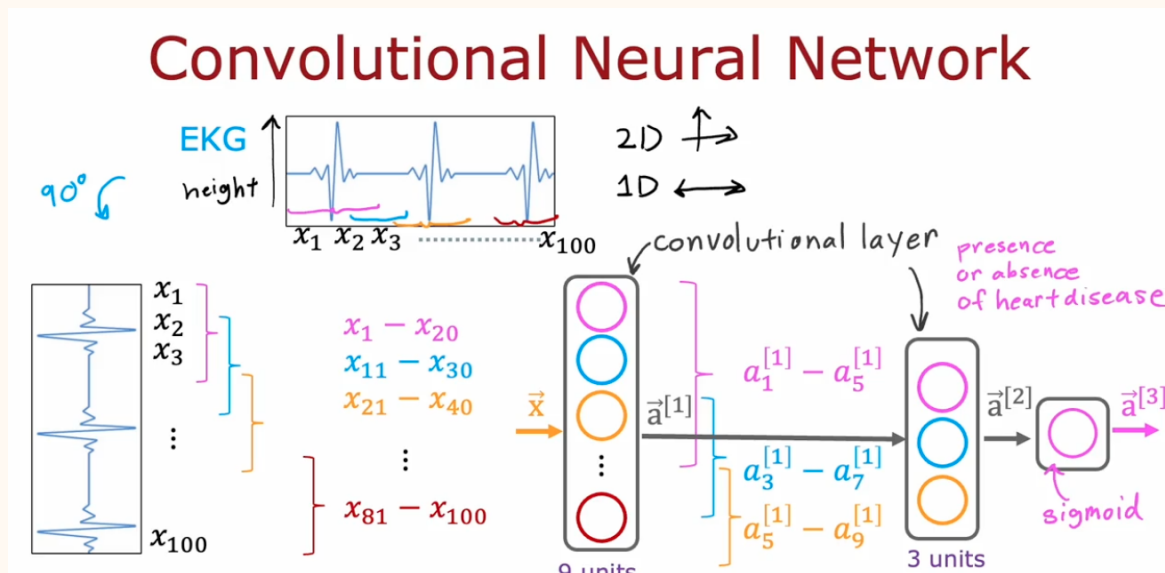
Definition

Convolution Layers are implemented where each neuron looks at different parts of the previous layer output as opposed to all the output. The benefits of this method is

- Faster computation
- Less training data needed (less prone to overfitting)

Example

Lets look at an EKG of a patient to see if they may have some heart condition. This can be done by a convolution hidden layer where layer one will look at sections of the ekg data and layer two will look at particular nodes of the first layer. The lecture slide below shows the breakdown as an example of the use of a convolutional neural network.



2.7 Implementation

Course Labs

- Multiclass Classification in tensorflow
 - Explore multiclass classification using neural networks in tensorflow using artificial data with scikit-learn `make_blobs`.
- ReLU activation
 - Explore how Relu activation works as a non-linear activation method for neural network

nodes.

- Softmax activation
 - Explore the implementation of softmax as nodes in neural networks and output handling

Graded Programming Assignment

- Neural Network for Multiclass Digit Recognition
 - Implement neural network with softmax layer for multiclass problem to recognize digits 0-9.

3. Week 3: Advice for applying machine learning

- Key objectives

- Neural Network Practical Advice
- Model evaluation and diagnostic tools

Note

- Steps that can be taken to debug a learning algorithm for large errors
 - More training examples
 - Smaller feature set (overfitting) or additional features (underfitting)
 - Additional polynomial features (underfitting)
 - decreasing/increasing parameters
- What is the most effective for your time to improve learning algorithm??

3.1 Model Evaluation

Definition

Training set is the subset of data to train the learning model **Testing set** is the subset of data used for model evaluation

If we have enough training data, we can split it into a ‘training set’ (x_{train}^k) and a ‘testing set’ (x_{test}^k). e.g., 70/30 split to test model performance on labeled data.

Formalism

– **Train/test procedure** for regularized linear regression with squared error cost:

- Fit parameters by minimizing cost function with only training subset:

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}_{train}^{(i)}) - y_{train}^{(i)})^2 \right]$$

- Compute the test error using the test subset (notice the regularization term is not included)

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2 \right]$$

- Also useful to compute the training error ($J_{test}(\vec{w}, b)$) using the above formula on the training subset

– If $J_{test}(\vec{w}, b)$ is high then the model is not generalizing well

– Train/test procedure for classification problem is the same as above replacing the cost function with logistic loss (course 1)

- Follow procedure above
- Additionally we can extract the fraction of test subset and train subset that is misclassified ($\hat{y} \neq y$), where here $J_{test}(\vec{w}, b)(J_{train}(\vec{w}, b))$ is exactly this fraction

3.2 Model Selection Automation

Definition

Generalization error is the error computed on the test set to evaluate the model predictive performance.

– From previous section we saw how $J_{test}(\vec{w}, b)$ is a much better indicator of model performance when evaluating on new data or for generalization.

- To choose a model you may want to test many models using the train/test procedure and take the model with the lowest $J_{test}(\vec{w}, b)$ (not recommended)
 - This is not recommended because when evaluating model performance the test error is likely an optimistic estimate of the generalization error since an extra parameter is needed when choosing the test set, e.g., the number of models
- For model selection automation we would want to modify the above procedure to account for the extra parameter.

Example

- This is done by splitting into **Training/cross validation/test** subsets, e.g., 60/20/20 split, where the cross validation set is used to check validity across different models.
- The Procedure then follows from above:
 - From the above test/train procedure example we now also include the cross validation (validation, dev) error:

$$J_{cv}(\vec{w}, b) = \frac{1}{2m_{cv}} \left[\sum_{i=1}^{m_{cv}} (f_{\vec{w}, b}(\vec{x}_{cv}^{(i)}) - y_{cv}^{(i)})^2 \right]$$

- compute model evaluation on the cross validation set instead of the test set, where we choose the model with the lowest $J_{cv}(\vec{w}, b)$.
- The we can estimate the generalization error using $J_{test}(\vec{w}, b)$

3.3 Bias and Variance

When diagnosing the learning algorithm performance the variance and bias of your model can guide you to the next steps to make it perform better.

Definition

High Bias typically refers to when your model does not fit the data well (underfit) with large errors and characterized by high $J_{train}(\vec{w}, b)$ and $J_{cv}(\vec{w}, b)$

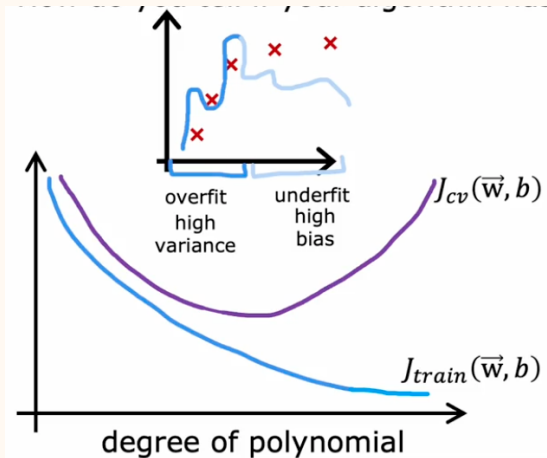
High Variance typically refers to when the model is quite constrained (overfit) where a model does not generalize well and characterized by low $J_{train}(\vec{w}, b)$ (model is fit well/overfit) but high $J_{cv}(\vec{w}, b)$ (model does not generalize to new data)

– To summarize the cases are:

- For high bias J_{train} is high ($J_{train} \propto J_{cv}$)
- For high variance $J_{cv} \gg J_{train}$ (J_{train} is typically low)
- High bias and high variance may occur when both conditions are met

Example

- For model evaluation with greater degree of polynomial terms we see that the J_{train} will continue to decrease because the increased flexibility to fit the data
- J_{cv} takes a conical shape representing the underfit models to the left of the minimum and overfit models to the right of the minimum.
- Using the cross validation set we can find the sweet spot for fitting and flexibility of our model



3.3.1 Regularized Models

Note

Reminder of the formalism for **regularization** from course 1 is to add a weighted term to the cost function to regulate the parameters or reduce them, e.g., linear regression $\frac{\lambda}{2m} \sum_{j=1}^n w_j^2$ where

- The regularization term λ penalized parameters in the cost function so to reduce them
- Very large $\lambda \gg 1$ would reduce the parameter to 0 $w_i \approx 0$
- Very small $\lambda \approx 0$ would essentially not effect the parameters

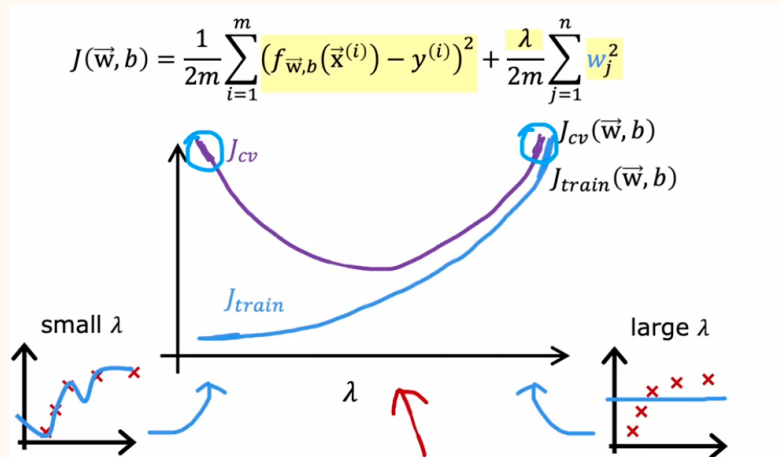
– Just like for model selection procedure we can do the same to find the optimal regularization term λ

- Again we try many lambda values and check the best one using the cross validation error
- Therefore we can get a good generalization error using the test subset error

Example

– Again we can draw a similar plot to above of the cost function with respect to the regularization term where we see

- The J_{train} increases now with increasing λ due to the parameters not being reduced and the model being overfit
- The J_{cv} now take a similar conical shape because the sweet spot for λ is when the model is not overfit or underfit and the parameters are properly regularized
- So the smaller λ correspond to high variance and the higher λ correspond to high bias



3.4 Establishing a baseline

When establishing a baseline we need to evaluate our model performance to a baseline performance. The learning model may be tested against:

- Human Level Performance
- Competing algorithm Performance
- educated guess based on experience

Example

An example using human voice recognition to evaluate a learning algorithm that recognizes speech may have these errors:

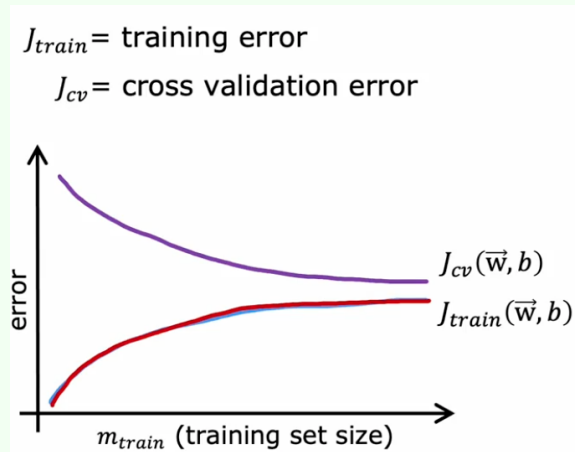
- Human performance error: 10.2%
 - Training set performance error: 10.4%
 - Cross validation set performance error: 14.5%
- From above if we did not have the human base line we may think the leaning algorithm has high bias because J_{train} looks large.
 - With the baseline performance we may not hope to do much better than this so 10.4% compared to the baseline 10.2% is reasonable.
 - Now we see that the learning model may have high variance because $J_{cv} \gg J_{train}$ and not generalizing well to new data

3.5 Learning curves

Revisiting learning curves from course 1, we remember that these curves tells us something about the performance of our learning algorithm.

Formalism

We can construct a learning curve of **error vs training size** to see how large our training set must be to get the desired performance. This is important because getting more data may be expensive.

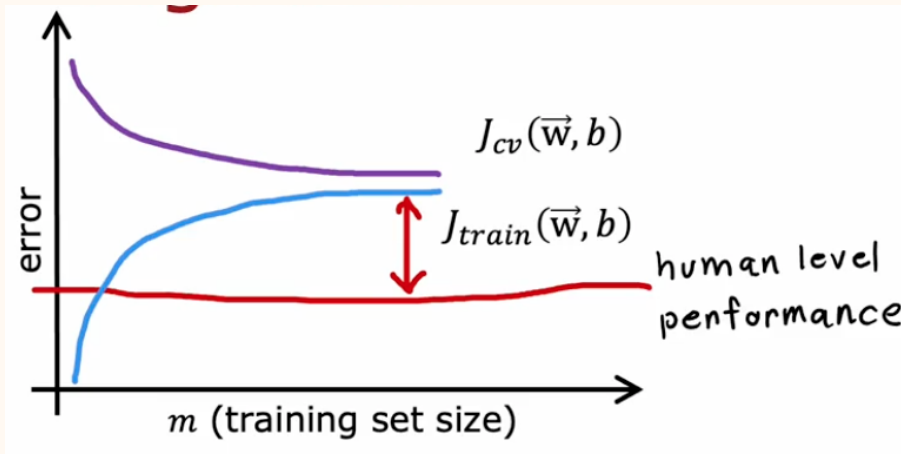


From the above learning curve we can draw a few conclusions. We see as the training size increases:

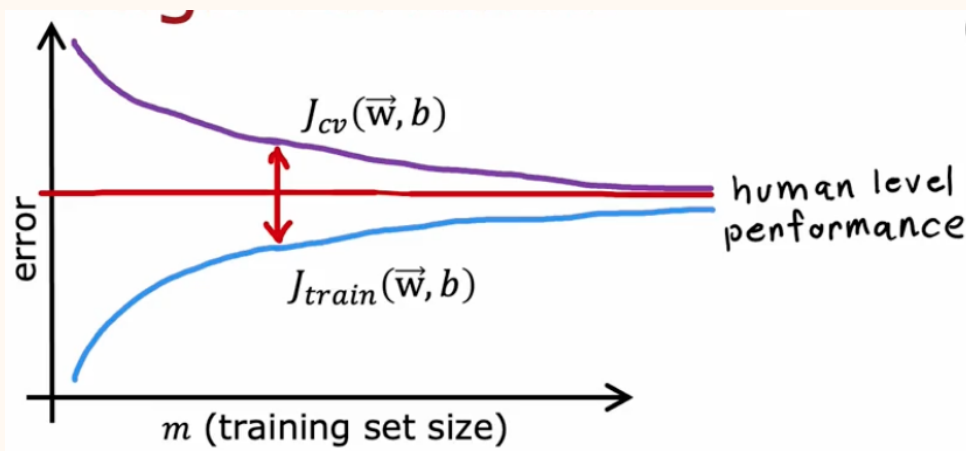
- J_{cv} curve decreases because the model needs to be more robust to fit more data
- J_{train} increases because there is more variability with more data and it is harder to fit
- Both of these curves plateau, this may be due to the data's behavior being encapsulated and the model no longer will change with more data

Example

– Lets look at an example of a learning curve that has **High Bias**, we see there is a large difference between the baseline and J_{train}



– Lets look at an example of a learning curve with **High Variance**, we see that the gap between the train error and cross validation error is large, but may be mitigated with larger training sample



Note

- From the learning curve behavior for high bias data, where the curves plateau with larger training sample, getting more training data will not improve our model
- From the learning curve behavior for high variance data, the curves converge to some value, so getting more training data will improve the model
- Downside of testing many subsets of data from bias and variance is that it can be computationally expensive

3.5.1 Revisit Model Diagnostic steps

In the introduction to this course we asked what steps are best to debug or improve the learning algorithm performance, now we are equipt to answer these questions.

Note

- Lets label the steps we outlined before with what situation would make this most useful for models that may have high bias or high variance

- More training examples (**high variance**)
- Smaller feature set (**high variance**) or additional features (**high bias**)
- Additional polynomial features (**high bias**)
- decreasing/increasing regularization parameter λ (**high bias/high variance**)

3.6 Bias/variance in neural networks

Note

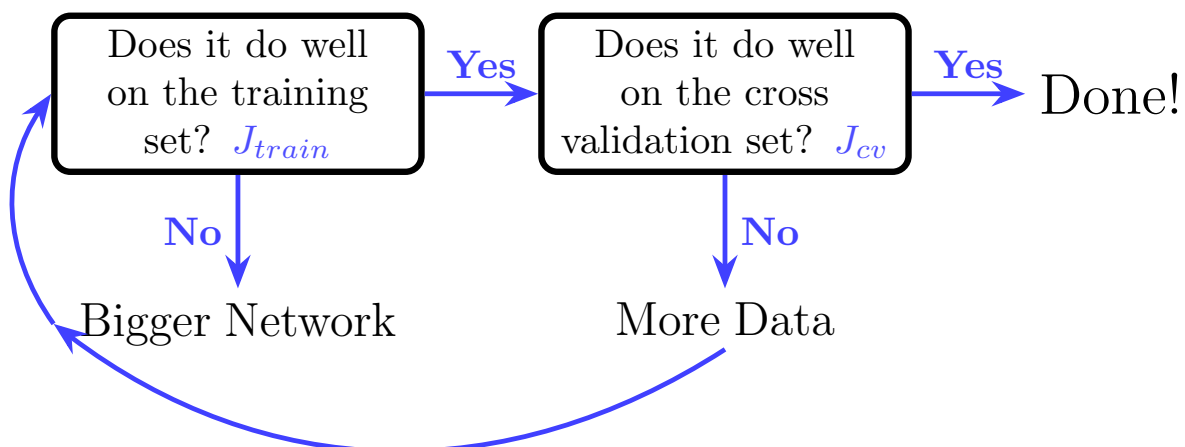
-Neural networks with large data have realigned how we evaluate high variance or high bias.

- Before neural network architectures, engineers had to tradeoff the degree of polynomials used (model complexity) with the regularization paramter λ to mitigate high variance/high bias

- Large Neural Networks with small or moderate data sets are low bias machine

- For a large enough neural network with small to moderate data sets the training sets are fit well (no tradeoff needed)

- Lets take at look at a simple recipe to



Where bigger networks mitigate high bias as explained and including more data mitigates high variance.

– The limitations of applying this procedure:

- Creating bigger neural networks is expensive or may need specialized equipment, e.g., gpus
- Obtaining more data may be expensive or impossible

Note

- Larger networks that are regularized appropriately generally perform better but need more compute
- Implementation of regularization in neural network looks like this

```
layer_1 = Dense(Units=25, activation='relu', kernel_regularizer=L2(0.01)))
```

3.7 Error Analysis

Definition

Error Analysis is the procedure to manually review misclassified/incorrect outputs from an ML model, where you may classify the errors into groups to diagnose best path forward(priority) for optimization

[Note] Error analysis can be very useful for problems that humans are good at

Example

- Lets take an ML algorithm used to filter spam emails, where the model is trained on the occurrence of the top 10000 words in the dictionary that are present in the email.
- If we misclassify 100 out of 500 emails from the cross validation set we can do an **error analysis** on these emails (For larger cv sets we can sample a set of the misclassified events for error analysis)
- To do the error analysis we manually view 100 samples and categorize them to identify where to improve the model for most benefit:
 - Deliberate misspellings
 - Unusual email routing
 - phishing emails
 - Pharma emails
- We can now focus on improving the areas of the model for the worst performers from our error analysis.

3.7.1 Adding data

Note

- With error analysis we may now be able to add data strategically to improve our models depending on the results of the error analysis.
- This is more efficient and targeted to improve the learning algorithm

Definition

Data augmentation used in image or voice recognition algorithms is a method to modify the existing data by some transformation without changing it to improve model performance or recognition

Synthetic Data is data that is artificially created for the purpose of training your models
Things to consider:

- This adds a more robust set of recognition by adding more training examples without needing to get more physical data.
- The augmentations should be representative of the type of noise/distortions in the test set and not random/meaningless noise

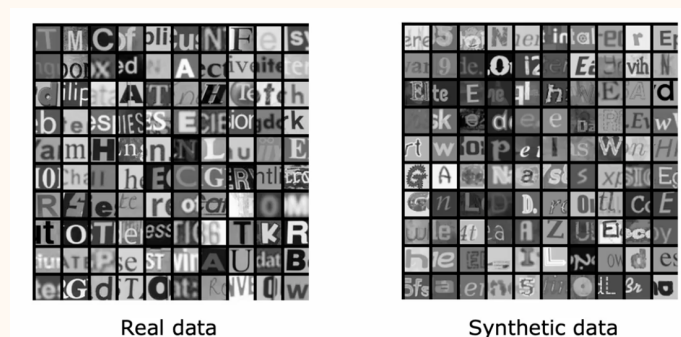
Example

Data augmentation (Data Engineering):

- For image recognition of letters you may augment the original image by making it larger, rotating it, taking the mirror image, etc
- May also place a grid on the image and distort it in different ways
- For speech data you may want to add background noise to original audio clips to identify the same audio in many different scenarios

Synthetic data:

- To recognize letters in an image(Photo OCR) one can generate synthetic data sets of letters of different fonts using a text editor



3.8 Transfer Learning

Definition

Transfer learning is a method to use data from an unrelated task to train a learning algorithm usually for neural networks

- Useful for when you have a limited set of data for a task you want a learning algorithm to perform

This is split into two steps:

1. **Supervised pretraining** is the first step where you train a neural network on some existing data to get some optimal parameters of the model
2. **Fine tuning** is the procedure of using the previous model parameters in a new neural network with a different output layer to fine-tune the parameters for the new task that is data limited

When performing the fine tuning procedure there are two options dependent on the available data for the new task:

1. Only train the output layer parameters in the new neural network using the parameters initialized in the pretraining (limited data)
2. Train all the parameters in the new neural network with the parameters of the pre-training model as initial values

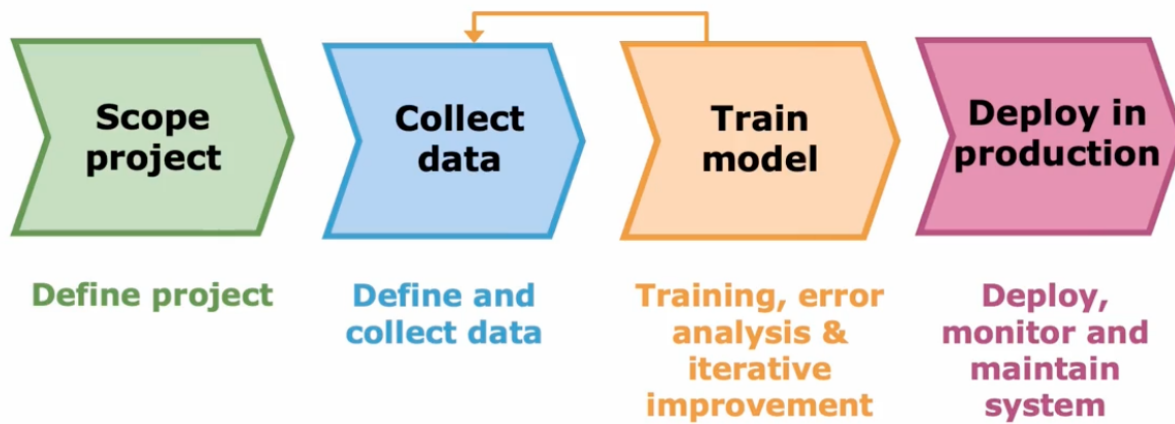
Note

- The idea here is that by using the parameters from a neural network with large data but unrelated task the parameters used in the new neural network with limited data will have learned already some important features to identify the new problem.
- This is very useful because parameters from existing neural networks that have already done the pretraining step can be used by only changing the output layer of your model
- Some intuition: Turns out that some of the layers of the pretrained neural network has learned to do generic things that are useful for other tasks, e.g., a pretrained neural network to identify images of cats, dogs, vehicles, etc has layers that are able to detect edges, corners, basic shapes, etc. that will be useful for recognizing digits (unrelated task)

Limitations:

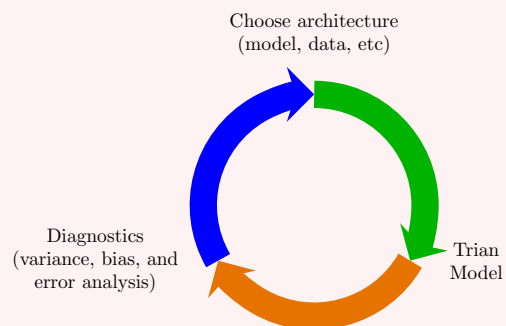
- The input format of your data must be the same
- The task to be done needs to be similar in nature

3.9 Full Cycle ML Project



Note

- We've focused on defining projects for machine learning algorithms and of data collection and engineering to train learning algorithms where the cycle of choosing and training learning algorithms is shown below:



- The deployment and management of learning algorithms for production applications is called **MLOps** (operations), e.g., engineering software to use ml models for apps or applications to serve users

3.9.1 Ethics

Definition

Bias in models takes the form of deployments that may:

- discriminate against women for hiring
- match dark skinned individuals to criminal mugshots in facial recognition
- biased loan approvals
- negative stereotyping

Adverse use cases (unethical):

- creating deepfakes using celebrity images without consent
- Using ML for spam or scam purposes

Fairness is usually referred to the open access to all people for the use, deployment and benefit of these machine learning algorithms

Note

General Guidelines for ethical deployment of learning algorithms:

- Having a diverse team for brainstorming possible harm for vulnerable groups
- Carry out literature search on standards/guidelines for the industry
- Audit system against possible harm prior to deployment
- Develop mitigation plan and monitor for harm after deployment

3.10 Handling skewed datasets (Optional)

3.11 Implementation

Course Labs

- Model Evaluation and Selection
 - Quantifying a learning algorithm's performance and comparing different models are some of the common tasks when applying machine learning to real world applications.
 - Split datasets into training, cross validation, test sets
 - evaluate regression and classification models
 - add polynomial features to improve model performance
 - compare several neural network architectures
- Diagnosing Bias and Variance
 - Use evaluation tools from previous lab and learn the signatures to diagnose high bias and high variance and methods to fix these issues(adding/reducing feature set, tuning

regularization, obtaining more training examples).

Graded Programming Assignment

- Bring together all the techniques and tools from weeks lab to evaluate and improve our machine learning models.

4. Week 4: Decision Trees

4.1 Overview

- Key objectives

- Decision tree models

- Application

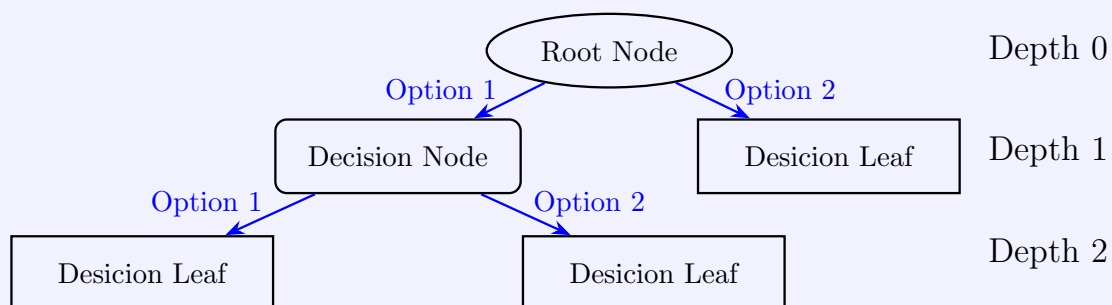
- Random Forest
- XGBoost

Definition

Decision trees are a diagrammatic representation of the splitting and decision making process of a learning algorithm or task, parts being

- The **root node** represents the top most feature.
- The **decision node** represents the next chosen feature as a result of the previous node option.
- The **decision leaf** represents the model evaluation from the given branch of events.
- The **depth** of the decision tree represents how many decisions are split
- The **purity** is how accurately a decision represents the group, e.g., if a decision categorizes 9/10 cats into a group its 90% pure

– General schematic of a decision tree:



Formalism

Decision Tree Learning consists of:

1. How to choose what feature to split at each node?
 - Maximize purity of the data
2. When to stop splitting?
 - When the node is 100% pure
 - When the tree exceeds a maximum depth
 - When improvements to purity or number of examples are below a threshold

4.2 Decision Tree Learning

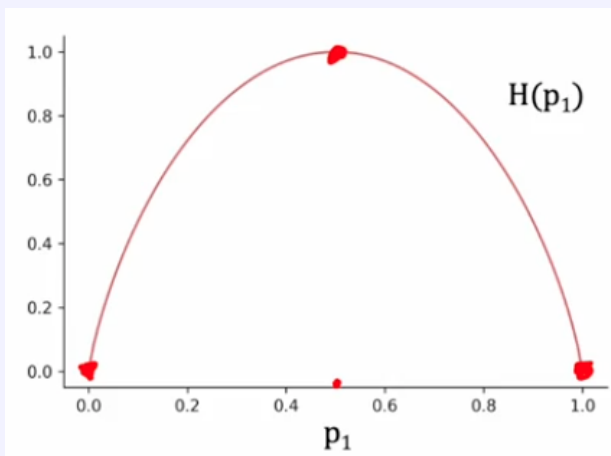
4.2.1 Measuring purity

Definition

Entropy $H(p_i)$ is the measurement of impurity from our decision tree, where p_i is the purity of the sample (ratio of correct categorization from $(0,1)$), defined as:

$$H(p_1) = -p_1 \log(p_1) - p_0 \log(p_0), \quad (9)$$

where $p_0 = 1 - p_1$ and below is the representation of the entropy function:



- For maximal impurity entropy is maximum $H(0.5) = 1$
- For maximal impurity/purity entropy is minimum $H(0, 1) = 0$

4.2.2 Choosing a split

Definition

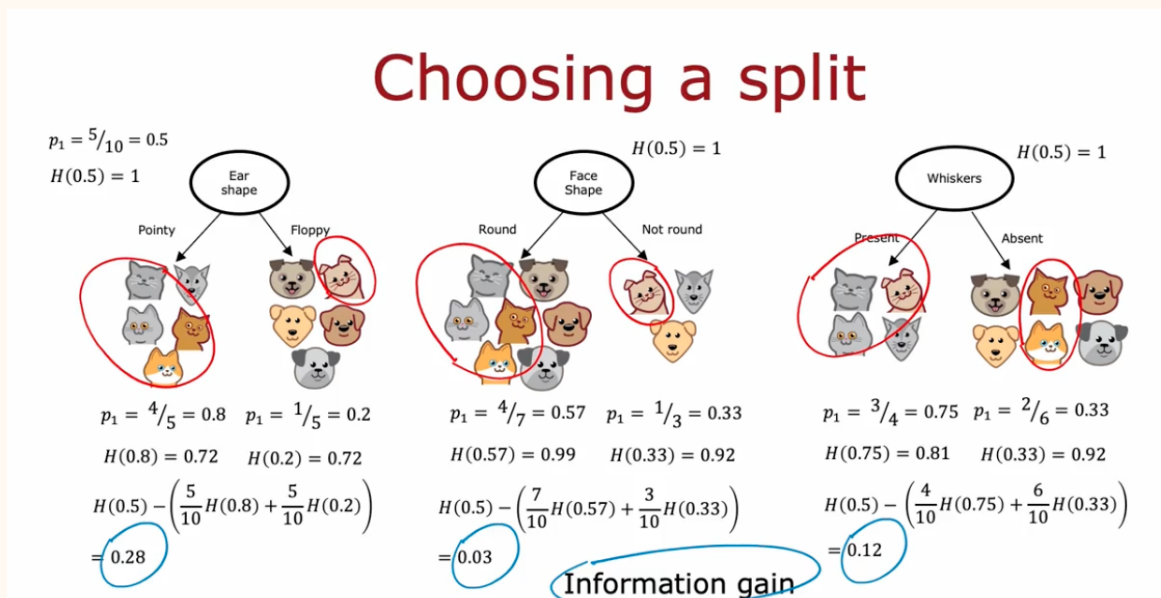
Information Gain is the reduction of the entropy function that helps decide how to split the data in our decision tree

Example

– Given a set of data with binary choices

{Earshape:(Pointy,Floppy), Face shape(round, not round), Whiskers(present,absent)}

for cat classification we can use the above formalism to identify the best features to split into:



where we see the calculated entropy values for the left and right subbranch for each chosen feature.

The procedure to reduce the entropy would be:

- Compute the entropy reduction of for each subbranch (**information gain**) by taking the weighted average subtracted by the root entropy of the root node

$$H_0(p_0) - \frac{1}{N} \sum_{i=1}^2 n_i H(p_i) \quad (10)$$

- This tells us the reduction of the entropy from the original root tree once split
- Can be used as a threshold to stop compute once reduction stops reducing

Note

- This algorithm is done recursively where for each subbranch Eq. (10) becomes the starting point for each subbranch until the stopping criteria is met and the decision tree is built.

4.2.3 Multiple output problems**Definition**

One-hot encoding for categorical features is a technique where we take a feature that take k values and create k binary features

- Can work for neural network algorithms and other learning algorithms

4.2.4 Continuous Features**Note**

- How do we split on continuous features
 - Many values along the continuous feature are tested split
 - Do the normal procedure for calculating information gain
 - Use the continuous value cutoff that maximizes the information gain
 - Example would be the weight of the animal for the cat classification problem

4.2.5 Regression Trees (optional)**Note**

- To output a number instead of a classification we use **regression trees**
 - Instead of computing the information gain we reduce the variance of the output continuous value
- Example: Using the similar cat features on the above problem we can predict the weight of the cat dependent on the features
 - The decision tree is the same as above but the outputs are the weights of the animals
 - We then calculate the variance of the data set for the root and sub branches (probably population):

$$s^2 = \frac{\sum (x_i - \mu)^2}{n - 1}$$

- Choose the splits that have the greatest reduction in variance of the root and sub branch (similar to information gain with variance)

4.3 Tree Ensembles

Definition

The **tree ensemble** is a collection of many different decision trees used to make the algorithm more robust

- Decision trees are not very robust on their own because changing the training set minimally leads to vastly different trees when doing the iterative information gain procedure.
- This is mitigated by using multiple decision trees and voting on the majority decision from the tree ensemble

Sampling with replacement is a technique used to build an ensemble of decision trees

- We make a new training set from the original by randomly sampling the training set with replacement
- When sampling with replacement we randomly choose a training example from the original training set without removing the chosen training example and repeat until we have a new training set of the same size as the original
- We can then make a new decision tree as part of our ensemble

Example

- We can generate a tree sample now in this way
 - Given a training set of size m , for $b = 1$ to B use sampling with replacement to create B new training sets to train B decision trees on the new data sets
 - Typically larger values of B improves the algorithm but have diminishing returns with the amount of needed compute
 - Recommended starting values may be $B = 64 \dots 128$
 - This procedure is called **Bagged decision tree algorithm** (putting all the trees in a bag)

4.3.1 Random Forest Algorithm

Definition

Random forest algorithm is an algorithm for building a tree ensemble by sampling with replacement where we randomize the feature choice to avoid making similar trees, e.g., without randomization we may get most of the trees with same root node

- At each node, when choosing a feature to use to split, if n features are available, pick a random subset $k < n$ features to choose from the information gain
- Typical choice for large feature sets for the number of subsets is $k = \sqrt{n}$ (probably statistically motivated because $\sqrt{N}$ is statistical error)
- This makes the algorithm robust/generalize because it trains on many differences in the original data set and averages over all these differences to make a decision.

4.3.2 Boosted Trees

Definition

Boosted trees is an algorithm to produce tree ensemble in the same way as the bagged decision tree algorithm but instead of choosing all examples with equal probability, make it more likely to pick misclassified examples from the previous trained trees

- After the first bagged decision tree we find the training examples that are misclassified and give them a higher weight to be chosen in the next iteration
- So the iterated decision trees focus on the data that it is not modeled well

Formalism

XGBoost(eXtreme Gradient Boosting) algorithm is an open source, fast efficient implementation of making boosted tree ensemble with good splitting criteria and regularization to prevent overfitting

- The mathematics is quite complex so for implementation typically libraries are used in this way (need to find paper to read)

```
from xgboost import XGBClassifier, XGBRegressor
```

```
model = XGBClassifier()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

- Replace above `model = XGBRegressor()` for regression algorithm instead of classification

4.4 Advice on choice of learning algorithm

Note

- Advantages/disadvantages of decision trees and tree ensembles
 - Works well on tabular (structured) data
 - Not recommended for unstructured data (audio, image, text)
 - Fast to train compared to neural networks
 - Small decision trees may be human interpretable (not too big)
- Advantages/disadvantages of neural networks
 - Works well on all type of data, structured and unstructured
 - May be slower to train
 - Works with transfer learning(critical for small data sets)
 - When building a system with multiple models, its easier to string together many neural networks

4.5 Implementations

Course Labs

- Decision trees
 - Visualize how a decision tree is splitting bases on information gain.
- Tree Ensembles
 - Perform one hot encoding using Pandas
 - Using Scikit-learn to implement a Decision Tree, Random Forest, and XGBoost models.

Graded Programming Assignment

- Bring together all the techniques and tools from weeks lab to evaluate and improve our machine learning models.